

第14章：指针和数组

目标：

1. 数组名的深刻理解和在代码中的使用
2. 掌握一维数组传参的本质
3. 掌握冒泡排序和优化
4. 掌握数组指针变量和二维数组传参的本质
5. 掌握指针数组
6. 能够使用指针数组模拟二维数组

1. 数组名的理解

1.1 认识数组名

在指针初阶部分，我们在使用指针访问数组的内容时，有这样的代码：

代码块

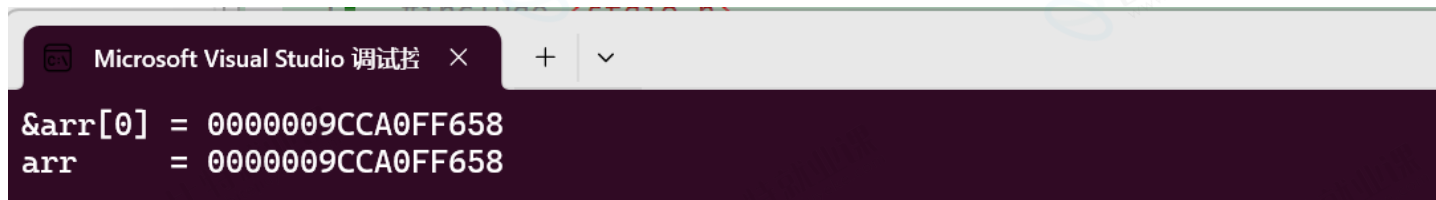
```
1  int arr[10] = {1,2,3,4,5,6,7,8,9,10};
2  int *p = &arr[0];
```

这里我们使用 `&arr[0]` 的方式拿到了数组第一个元素的地址，但是其实数组名也是地址，是数组首元素的地址，我们来做个测试。

代码块

```
1  #include <stdio.h>
2  int main()
3  {
4      int arr[10] = { 1,2,3,4,5,6,7,8,9,10 };
5      printf("&arr[0] = %p\n", &arr[0]);
6      printf("arr      = %p\n", arr);
7      return 0;
8  }
```

输出结果：



我们发现数组名和数组首元素的地址打印出的结果一模一样，**数组名就是数组首元素(第一个元素)的地址**。

这时候有同学会有疑问？数组名如果是数组首元素的地址，那下面的代码怎么理解呢？

代码块

```
1  #include <stdio.h>
2  int main()
3  {
4      int arr[10] = { 1,2,3,4,5,6,7,8,9,10 };
5      printf("%d\n", sizeof(arr));
6      return 0;
7  }
```

输出的结果是：40，如果 `arr` 是数组首元素的地址，那输出应该的应该是 4 或者 8 才对呀。

其实数组名就是数组首元素(第一个元素)的地址是对的，但是有两个例外：

- **sizeof(数组名)**，sizeof中单独放数组名，这里的数组名表示整个数组，计算的是整个数组的大小，单位是字节
- **&数组名**，这里的数组名表示整个数组，取出的是**整个数组的地址**（整个数组的地址和数组首元素的地址虽然值是一样的，但是类型不同）

除去这两种场景之外，任何地方使用数组名，数组名都表示首元素的地址。

这时有好奇的同学，再试一下这个代码：

代码块

```
1  #include <stdio.h>
2  int main()
3  {
4      int arr[10] = { 1,2,3,4,5,6,7,8,9,10 };
5      printf("&arr[0] = %p\n", &arr[0]);
6      printf("arr      = %p\n", arr);
7      printf("&arr    = %p\n", &arr);
8      return 0;
}
```

```
9 }
```

测试输出的结果：

代码块

```
1 &arr[0] = 000000EEAA0FF8E8
2 arr    = 000000EEAA0FF8E8
3 &arr    = 000000EEAA0FF8E8
```

三个打印结果一模一样，这时候又纳闷了，那arr和&arr有啥区别呢？

代码块

```
1 #include <stdio.h>
2 int main()
3 {
4     int arr[10] = { 1,2,3,4,5,6,7,8,9,10 };
5     printf("&arr[0]    = %p\n", &arr[0]);
6     printf("&arr[0]+1 = %p\n", &arr[0]+1);
7     printf("arr      = %p\n", arr);
8     printf("arr+1    = %p\n", arr+1);
9     printf("&arr     = %p\n", &arr);
10    printf("&arr+1   = %p\n", &arr+1);
11    return 0;
12 }
```

输出结果：

代码块

```
1 &arr[0]    = 000000EE7CFF648 //int*
2 &arr[0]+1  = 000000EE7CFF64C
3 arr       = 000000EE7CFF648 //int*
4 arr+1     = 000000EE7CFF64C
5 &arr      = 000000EE7CFF648 //int(*)[10]
6 &arr+1    = 000000EE7CFF670
```

这里我们发现 `&arr[0]` 和 `&arr[0]+1` 相差 4 个字节，`arr` 和 `arr+1` 相差4个字节，是因为 `&arr[0]` 和 `arr` 都是首元素的地址，`+1` 就是跳过一个元素，也就是4个字节。

但是 `&arr` 和 `&arr+1` 相差40个字节，这就是因为 `&arr` 是数组的地址，`+1` 操作是跳过整个数组的。

到这里我们应该就分析清楚数组名在不同代码场景下的意思了。

- **sizeof(数组名)**，这里的数组名表示整个数组，计算的是整个数组的大小，单位是字节
- **&数组名**，这里的数组名表示整个数组，取出的是**整个数组的地址**

除去这两种场景之外，任何地方使用数组名，数组名都表示首元素的地址

当数组名表示数组首元素的地址时，地址是一个常量的值，是不能修改的，所以下面代码中的赋值就是非法的。

代码块

```
1  int arr1[5] = {1,2,3,4,5};
2  int arr2[5] = {0};
3  arr1 = arr2; //err
```

1.2 重新认识指针访问数组

这里我们重新梳理一下之前学习过的使用指针访问一维数组的代码：

代码块

```
1  #include <stdio.h>
2  int main()
3  {
4      int arr[10] = {1,2,3,4,5,6,7,8,9,10};
5      int *p = &arr[0]; //获得第一个元素的地址
6      int i = 0;
7      int sz = sizeof(arr)/sizeof(arr[0]);
8      for(i = 0; i < sz; i++)
9      {
10         printf("%d ", *(p + i)); //p+i是下标为i这个元素的地址
11     }
12     return 0;
13 }
```

上面代码中第5行的代码，就可以换成：

代码块

```
1  #include <stdio.h>
```

```

2  int main()
3  {
4      int arr[10] = {1,2,3,4,5,6,7,8,9,10};
5      int *p = arr; //获得第一个元素的地址
6      int i = 0;
7      int sz = sizeof(arr)/sizeof(arr[0]);
8      for(i = 0; i < sz; i++)
9      {
10         printf("%d ", *(p + i)); //p+i是下标为i这个元素的地址
11     }
12     return 0;
13 }

```

那么我们再分析一下，`arr` 是首元素的地址，赋值给了 `p`，`p` 变量中存放的也是首元素的地址，这里的 `arr` 和 `p` 是等价的。那么我们可以把上面代码中第10行的 `p` 换成 `arr`，代码如下：

代码块

```

1  #include <stdio.h>
2  int main()
3  {
4      int arr[10] = {1,2,3,4,5,6,7,8,9,10};
5      int *p = arr; //获得第一个元素的地址
6      int i = 0;
7      int sz = sizeof(arr)/sizeof(arr[0]);
8      for(i = 0; i < sz; i++)
9      {
10         printf("%d ", *(arr + i));
11         //arr+i是下标为i这个元素的地址
12         //*(arr + i) 就是下标i的元素，也就是arr[i]
13     }
14     return 0;
15 }

```

代码中的 `*(arr + i)` 就是下标 `i` 的元素，也就是 `arr[i]`。

那么我们可以推出：`*(p + i)` 等价于 `*(arr + i)` 等价于 `arr[i]` 等价于 `p[i]`

在访问一个数组元素的时候，数组下标的写法和指针的写法可以互相转换的。其实写成 `arr[i]`，编译器也要转换成 `*(arr+i)` 的方式来处理的。那这两种写法哪种好点呢？

- `arr[i]` 是语法糖，简洁直观，不容易出错
- 现代编译器对这两种写法生成的机器码完全相同，因此不需要为了“效率”而刻意写指针形式，除非处于特殊的底层编程（如嵌入式、手写汇编）。

1.3 再谈字符指针

在指针的类型中我们知道有一种指针类型为字符指针 `char*`;

一般使用:

代码块

```
1  #include <stdio.h>
2  int main()
3  {
4      char ch = 'w';
5      char *pc = &ch;
6      *pc = 'w';
7      return 0;
8  }
```

字符指针指向常量字符串:

代码块

```
1  #include <stdio.h>
2  int main()
3  {
4      const char* pstr = "hello bit";//这里是把一个字符串放到pstr指针变量里了吗?
5      printf("%s\n", pstr);
6      return 0;
7  }
```

我们可以通俗的理解为 `pstr` 指针变量, 指向了一个常量字符串"hello bit"。

常量字符串和字符数组很相似, 也可以按照下标来访问, 但是常量字符串是只读的, 也就是不能被修改。

数组可以放在内存的栈区、静态区, 常量字符串是放在内存的只读数据区的, 如果修改常量字符串程序会崩溃。

| `const` 关键字会在《指针的最佳实践》部分讲解

代码块

```
1  #include <stdio.h>
2
3  int main()
4  {
5      const char* pstr = "hello bit";//这里是把一个字符串放到pstr指针变量里了吗?
6      printf("%c\n", pstr[1]);
7      printf("%c\n", "hello bit"[1]);
```

```
8     return 0;
9 }
```



代码 `const char* pstr = "hello bit";` 特别容易被理解为：是把字符串 `hello bit` 放到字符指针 `pstr` 里了，但是本质是把字符串 `hello bit` 首字符 `'h'` 的地址放到了 `pstr` 中。



上面代码的意思是把一个常量字符串的首字符 `h` 的地址存放到指针变量 `pstr` 中。

《剑指offer》中收录了一道和字符数组和常量字符串相关的笔试题，我们一起来学习一下：

代码块

```
1  #include <stdio.h>
2
3  int main()
4  {
5      char str1[] = "hello bit.";
6      char str2[] = "hello bit.";
7      const char *str3 = "hello bit.";
8      const char *str4 = "hello bit.";
9
10     if(str1 == str2)
11         printf("str1 and str2 are same\n");
```



```

12     else
13         printf("str1 and str2 are not same\n");
14
15     if(str3 ==str4)
16         printf("str3 and str4 are same\n");
17     else
18         printf("str3 and str4 are not same\n");
19
20     return 0;
21 }

```

Microsoft Visual Studio 调试器 × + ▾

```

str1 and str2 are not same
str3 and str4 are same

```

这里 `str3` 和 `str4` 指向的是一个同一个常量字符串。C/C++会把常量字符串存储到单独的一个内存区域，当几个指针指向同一个常量字符串的时候，他们实际会指向同一块内存。但是用相同的常量字符串去初始化不同的数组的时候就会开辟出不同的内存块。所以 `str1` 和 `str2` 不同，`str3` 和 `str4` 相同。

1.4 练习

练习1：程序运行的结果是啥？

代码块

```

1  #include <stdio.h>
2
3  int main()
4  {
5      int a[] = {1,2,3,4};
6      printf("%zu\n",sizeof(a));
7      printf("%zu\n",sizeof(a+0));
8      printf("%zu\n",sizeof(*a));
9      printf("%zu\n",sizeof(&a));
10     printf("%zu\n",sizeof(*&a));
11     printf("%zu\n",sizeof(&a[0]+1));
12     return 0;
13 }

```

练习2：程序运行的结果是啥？

代码块


```

1  #include <stdio.h>
2
3  int main()
4  {
5      int a[5] = { 1, 2, 3, 4, 5 };
6      int *ptr = (int *)(&a + 1);
7      printf( "%d,%d", *(a + 1), *(ptr - 1));
8      return 0;
9  }

```

a数组

1	2	3	4	5
---	---	---	---	---

2. 一维数组传参和指针

在学习数组的时候，我们就可以把一维数组作为参数传递给函数，假设我们要写一个函数打印数组，代码如下：

代码块

```

1  #include <stdio.h>
2  void Print(int arr[], int sz)
3  {
4      int i = 0;
5      for (i = 0; i < sz; i++)
6      {
7          printf("%d ", arr[i]);
8      }
9      printf("\n");
10 }
11
12 int main()
13 {
14     int arr[10] = { 1,2,3,4,5,6,7,8,9,10 };
15     int sz = sizeof(arr) / sizeof(arr[0]);
16     Print(arr, sz);

```

```
17     return 0;
18 }
```

这里我们设计的 `Print` 函数的参数写成了数组的形式，数组传参，形参写成数组的形式，非常直观、容易理解。

但是当我们理解了数组名其实是数组首元素的地址时，我们继续往下想，上面的代码中第 16 行，调用 `Print(arr, sz)` 函数时传递的数组名 `arr` 就是数组首元素的地址，那形参应该写成指针类型也应该是正确的吧？对的，**这就是一维数组传参的本质：传递的是首元素的地址。**

所以代码也可以这么写：

代码块

```
1  #include <stdio.h>
2  void Print(int *arr, int sz)
3  {
4      int i = 0;
5      for (i = 0; i < sz; i++)
6      {
7          printf("%d ", *(arr+i));
8      }
9      printf("\n");
10 }
11
12 int main()
13 {
14     int arr[10] = { 1,2,3,4,5,6,7,8,9,10 };
15     int sz = sizeof(arr) / sizeof(arr[0]);
16     Print(arr, sz);
17     return 0;
18 }
```

既然数组传参传递的是数组首元素的地址，那么**形参的指针找到数组和实参的数组就是同一个数组**了，这样我们就彻底理解了，为什么我们在学习函数章节的时候，说：一维数组传参的时候，形参不会创建新的数组，形参数组和实参数组是同一个数组了。

? 那函数传参的时候形参建议写数组的形式？还是指针的形式呢？

我的建议是写成数组的形式，代码可读性更好，更容易维护。

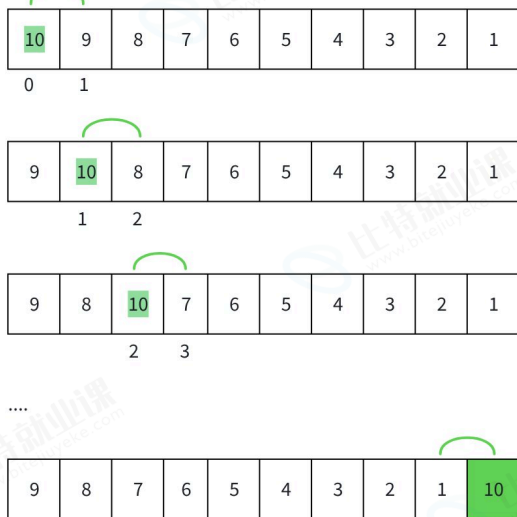
在现代的编译器上，其实写成指针的形式和数组的形式性能差异不大。

3. 冒泡排序

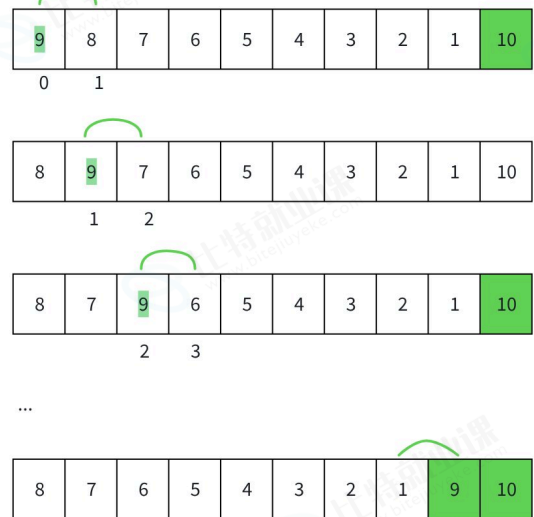
冒泡排序（Bubble Sort）是最基础的排序算法之一。它的核心思想很简单：重复遍历要排序的列表，依次比较相邻的两个元素，如果顺序错误就交换它们。每一轮遍历都会把当前未排序部分中最大（或最小）的元素“浮”到顶端，就像气泡一样上浮，因此得名。

现在有一组数据：10 9 8 7 6 5 4 3 2 1，需要对数据进行排序，目标是升序

第1趟冒泡排序



第2趟冒泡排序



代码块

```
1  #include <stdio.h>
2  //方法1
3  void bubble_sort(int arr[], int sz)//参数接收数组元素个数
4  {
5      int i = 0;
6      for(i = 0; i < sz-1; i++)
7      {
8          int j = 0;
9          for(j = 0; j < sz-i-1; j++)
10         {
11             if(arr[j] > arr[j+1])
12             {
13                 int tmp = arr[j];
14                 arr[j] = arr[j+1];
15                 arr[j+1] = tmp;
16             }
17         }
18     }
19 }
20
```

```

21  int main()
22  {
23      int arr[] = {10,9,8,7,6,5,4,3,2,1}; //{3,1,7,5,8,9,0,2,4,6};
24      int sz = sizeof(arr)/sizeof(arr[0]);
25      bubble_sort(arr, sz);
26      int i = 0;
27      for(i = 0; i < sz; i++)
28      {
29          printf("%d ", arr[i]);
30      }
31      return 0;
32  }

```

我们写的这个冒泡排序2层for循环，循环次数是按照元素个数推算出来的。

但是如果待排序的数据是这样的：

代码块

```

1  9 8 7 6 10 5 4 3 2 1
2  经过第1趟排序后：
3  9 8 7 6 5 4 3 2 1 10 //此时已经有序了，但是程序不知道
4  进行第2趟排序时：相邻的元素都不会发生交换，就说明已经有序了，就没必要继续循环处理了

```

所以上面的代码是有优化空间的，尤其是在数据接近有序的情况下，优化代码如下：

代码块

```

1  #include <stdio.h>
2  void bubble_sort(int arr[], int sz)//参数接收数组元素个数
3  {
4      int i = 0;
5      for(i = 0; i < sz-1; i++)
6      {
7          int flag = 1;//假设这一趟已经有序了
8          int j = 0;
9          for(j = 0; j < sz-i-1; j++)
10         {
11             if(arr[j] > arr[j+1])
12             {
13                 flag = 0;//发生交换就说明，无序
14                 int tmp = arr[j];
15                 arr[j] = arr[j+1];
16                 arr[j+1] = tmp;
17             }
18         }

```

```

19         if(flag == 1)//这一趟没交换就说明已经有序，后续无序排序了
20             break;
21     }
22 }
23
24 int main()
25 {
26     int arr[] = {9,8,7,6,10,5,4,3,2,1};
27     int sz = sizeof(arr)/sizeof(arr[0]);
28     bubble_sort(arr, sz);
29     int i = 0;
30     for(i = 0; i < sz; i++)
31     {
32         printf("%d ", arr[i]);
33     }
34     return 0;
35 }

```

4. 数组指针变量

4.1 数组指针变量是什么？

数组指针变量是指针变量？还是数组？

答案是：指针变量。

我们已经熟悉：

- **整形指针变量：** `int* pint;` 存放的是整形变量的地址，能够指向整形数据的指针。
- **浮点型指针变量：** `float* pf;` 存放浮点型变量的地址，能够指向浮点型数据的指针。

那数组指针变量应该是：存放的应该是数组的地址，能够指向数组的指针变量。

举例如下：

代码块

```
1  int (*p)[10];
```

解释： `p` 先和 `*` 相遇，说明 `p` 是一个指针变量，然后指针指向的是一个大小为10个元素的数组，数组的每个元素是int类型，所以 `p` 是一个指针，指向一个数组，叫 **数组指针变量**。

这里要注意： `[]` 的优先级要高于 `*` 号的，所以必须加上 `()` 来保证 `p` 先和 `*` 结合。

4.2 数组指针变量初始化

数组指针变量是用来存放数组地址的，那怎么获得数组的地址呢？就是我们之前学习的 `&数组名`。

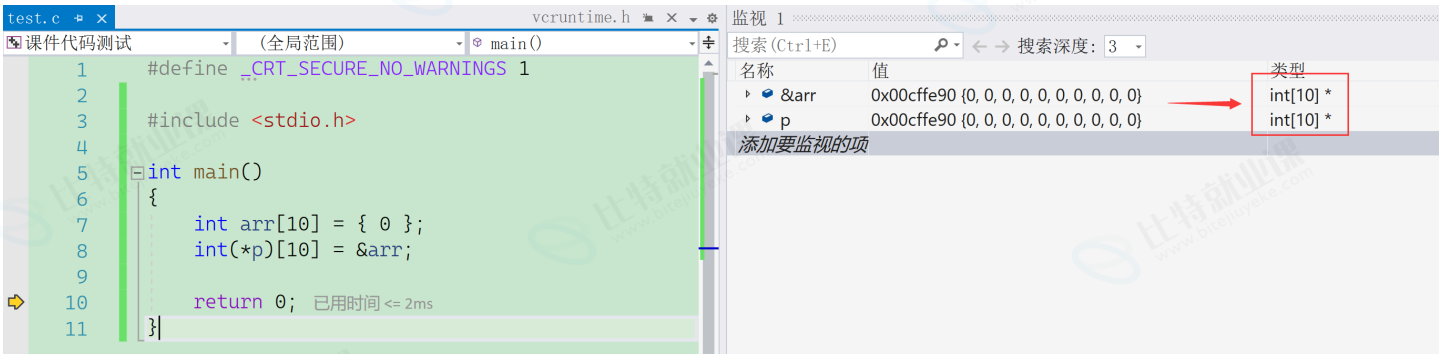
代码块

```
1  int arr[10] = {0};
2  &arr; //得到的就是数组的地址
```

如果要存放个数组的地址，就得存放在**数组指针变量**中，如下：

代码块

```
1  int(*p)[10] = &arr;
```



我们调试也能看到 `&arr` 和 `p` 的类型是完全一致的。

数组指针类型解析：

代码块

```
1  int (* p) [10] = &arr;
2  |      |      |
3  |      |      |
4  |      |      p指向数组的元素个数
5  |      p是数组指针变量名
6  p指向的数组的元素类型
```

5. 二维数组传参和指针

有了数组指针的理解，我们就能讲一下二维数组传参和指针的关系。

过去二维数组需要传参给一个函数的时候时，我们是这样写的：

代码块

```
1  #include <stdio.h>
```

```

2
3 void test(int a[3][5], int r, int c)
4 {
5     int i = 0;
6     int j = 0;
7     for(i = 0; i < r; i++)
8     {
9         for(j = 0; j < c; j++)
10        {
11            printf("%d ", a[i][j]);
12        }
13        printf("\n");
14    }
15 }
16
17 int main()
18 {
19     int arr[3][5] = {{1,2,3,4,5}, {2,3,4,5,6},{3,4,5,6,7}};
20     test(arr, 3, 5);
21     return 0;
22 }

```

这里实参是二维数组，形参也写成二维数组的形式，那还有什么其他的写法吗？毕竟二维数组传参，实参也写的是数组名，数组名是数组首元素的地址呀。那形参能写成指针吗？

首先我们再次理解一下二维数组，二维数组其实可以看做是每个元素是一维数组的数组，也就是二维数组的每个元素是一个一维数组。那么二维数组的首元素就是第一行，是个一维数组。

如下图：

	0	1	2	3	4	
0	1	2	3	4	5	
1	2	3	4	5	6	
2	3	4	5	6	7	
	arr数组					

所以，根据数组名是数组首元素的地址这个规则，二维数组的数组名表示的就是第一行的地址，是一维数组的地址。根据上面的例子，第一行的一维数组的类型就是 `int [5]`，所以第一行的地址的类型就是数组指针类型 `int(*)[5]`。那就意味着**二维数组传参本质上也是传递了地址，传递的是第一行这个一维数组的地址**，那么形参也是可以写成指针形式的。如下：


```

1 代码块 include <stdio.h>
2
3  void test(int (*p)[5], int r, int c)
4  {
5      int i = 0;
6      int j = 0;
7      for(i = 0; i < r; i++)
8      {
9          for(j = 0; j < c; j++)
10         {
11             printf("%d ", *(*(p+i)+j));
12             //p+i 是第i行的地址
13             //*(p+i) == p[i], 是第i行的数组名
14             //数组名是首元素的地址, 所以*(p+i) 也是第i行第一个元素的地址
15             //*(p+i)+j 就是第i行, 下标为j的元素的地址
16             //*(*(p+i)+j)就是i行, j列的元素
17             //*(*(p+i)+j) == p[i][j]
18         }
19         printf("\n");
20     }
21 }
22
23 int main()
24 {
25     int arr[3][5] = {{1,2,3,4,5}, {2,3,4,5,6},{3,4,5,6,7}};
26     test(arr, 3, 5);
27     return 0;
28 }

```

总结：二维数组传参，形参的部分可以写成二维数组，也可以写成数组指针的形式。

练习：

```

1 代码块
2 #include <stdio.h>
3 int main()
4 {
5     int aa[2][5] = { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 };
6     int *ptr1 = (int *)(&aa + 1);
7     int *ptr2 = (int *)(*(&aa + 1));
8     printf( "%d,%d", *(ptr1 - 1), *(ptr2 - 1));
9     return 0;
10 }

```

aa数组

1	2	3	4	5
6	7	8	9	10

6. 指针数组

指针数组是指针还是数组？

我们类比一下，整型数组，是存放整型的数组，字符数组是存放字符的数组。

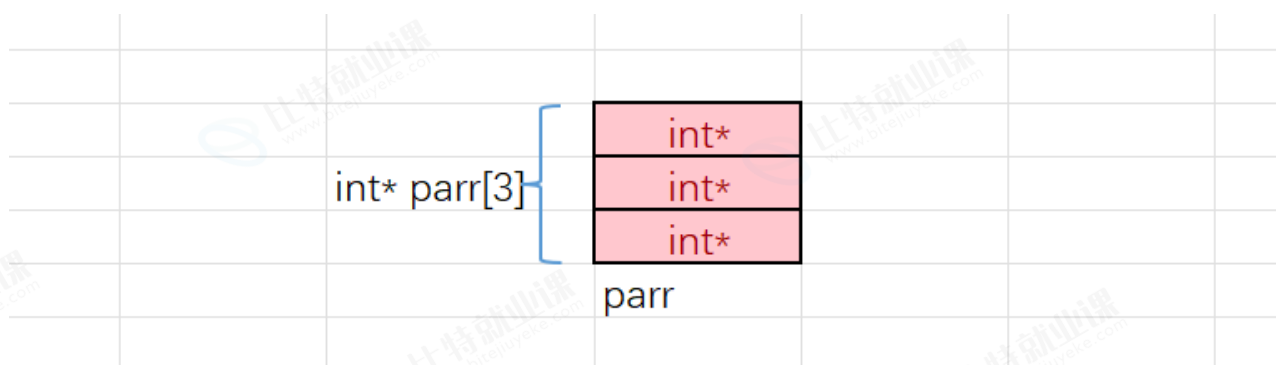
那指针数组呢？是存放指针的数组。



整型数组和字符数组

指针数组的每个元素都是用来存放地址（指针）的。

如下图：



`parr` 是一个数组，数组有3个元素，每个元素的类型是 `int*`，`parr` 就是指针数组。

指针数组的每个元素是地址，又可以指向一块区域。

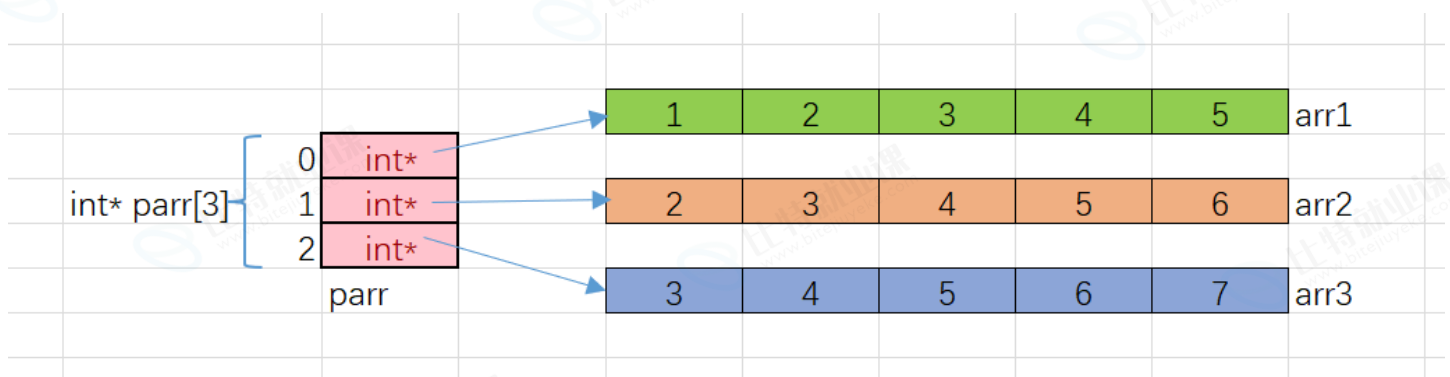
7. 指针数组模拟二维数组

代码块

```

1  #include <stdio.h>
2  int main()
3  {
4      int arr1[] = {1,2,3,4,5};
5      int arr2[] = {2,3,4,5,6};
6      int arr3[] = {3,4,5,6,7};
7      //数组名是数组首元素的地址，类型是int*的，就可以存放在parr数组中
8      int* parr[3] = {arr1, arr2, arr3};
9      int i = 0;
10     int j = 0;
11     for(i = 0; i < 3; i++)
12     {
13         for(j = 0; j < 5; j++)
14         {
15             printf("%d ", parr[i][j]);
16         }
17         printf("\n");
18     }
19     return 0;
20 }

```



parr数组的画图演示

`parr[i]` 是访问 `parr` 数组的元素，`parr[i]` 找到的数组元素指向了整型一维数组，`parr[i][j]` 就是整型一维数组中的元素。

上述的代码模拟出二维数组的效果，实际上并非完全是二维数组，因为每一行并非是连续的。

练习1: 下面代码输出的结果是啥？

代码块

```

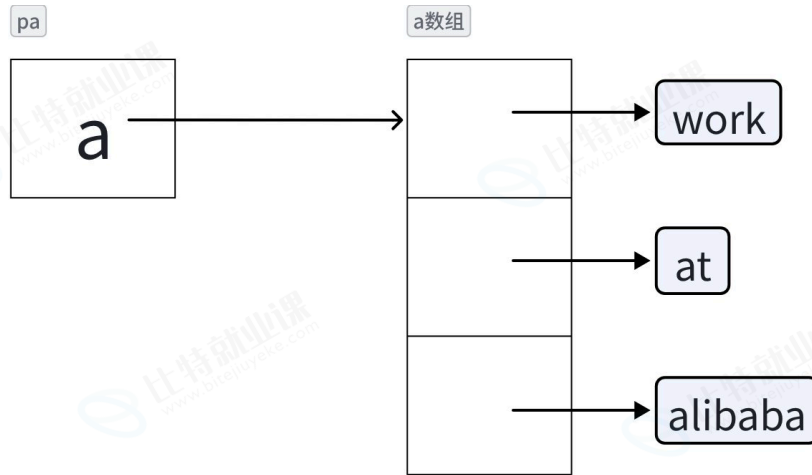
1  #include <stdio.h>
2  int main()
3  {
4      char *a[] = {"work","at","alibaba"};
5      char**pa = a;

```

```

6     pa++;
7     printf("%s\n", *pa);
8     return 0;
9 }

```



练习2: (加餐)

代码块

```

1  #include <stdio.h>
2
3  int main()
4  {
5      char *c[] = {"ENTER", "NEW", "POINT", "FIRST"};
6      char**cp[] = {c+3, c+2, c+1, c};
7      char***cpp = cp;
8      printf("%s\n", *+++cpp);
9      printf("%s\n", *--+++cpp+3);
10     printf("%s\n", *cpp[-2]+3);
11     printf("%s\n", cpp[-1][-1]+1);
12     return 0;
13 }

```

